

GUÍA DE LABORATORIO N.º 2

Construcción de un datamart analítico y ejecución de operaciones OLAP

1. Datos generales

Tema	Diseño de datamarts para DWH/DM analítico, herramientas OLAP y metodologías de construcción
Modalidad	INDIVIDUAL. Cada estudiante desarrolla y entrega su propio trabajo.

1.1. Objetivos

Al concluir este laboratorio, el estudiante será capaz de:

- Perfilar un conjunto de datos de origen e identificar problemas de calidad que afectan el diseño del datamart.
- Aplicar el método de cuatro pasos de Kimball para derivar un esquema estrella a partir de fuentes transaccionales reales.
- Construir dimensiones con clave subrogada, fila de valor desconocido y atributos conformados.
- Cargar una tabla de hechos al grano atómico y verificar la integridad de la carga mediante cuadros de control.
- Ejecutar en SQL las operaciones OLAP de roll-up, drill-down, slice, dice, pivot y drill-across.

1.2. Requisitos previos

- Haber revisado el material teórico de la Semana 2, en particular el método de cuatro pasos y la tipología de dimensiones.
- Conocimientos de SQL a nivel de la asignatura Base de Datos II: JOIN, GROUP BY, subconsultas y funciones de agregación.
- Computadora con Python 3.10 o superior instalado y acceso a Internet para la instalación de un paquete.

1.3. Entregables

Cada estudiante sube al Aula Virtual, en un único archivo comprimido con el nombre Lab02_ApellidoNombre.zip, lo siguiente:

N.º	Entregable	Descripción
1	Script SQL o Python	Archivo lab02_datamart.sql o lab02_datamart.py con todas las sentencias ejecutadas, en orden y comentadas.
2	Informe en PDF	Máximo 6 páginas: hoja de diseño del paso 3, capturas de los cuadros de control y de los resultados de las consultas OLAP, y respuestas a las preguntas de análisis.
3	Actividad propuesta	Script y resultados de la sección 10, incluidos en el mismo informe como sección final.

Plazo de entrega: hasta el inicio de la sesión de la Semana 3.

2. Herramienta y datos de trabajo

2.1. ¿Por qué DuckDB?

El laboratorio se desarrolla sobre DuckDB, un motor analítico de código abierto, columnar y orientado a procesamiento OLAP. Se eligió por tres razones prácticas: se instala con un solo comando y sin servidor, lee archivos CSV directamente sin necesidad de un proceso de importación previo, y soporta el SQL analítico completo que necesitaremos (ROLLUP, CUBE, GROUPING SETS, PIVOT y funciones de ventana). Todo lo que usted escriba aquí es SQL estándar y se traslada casi sin cambios a PostgreSQL, SQL Server o Snowflake.

Alternativa: si usted prefiere trabajar en MySQL o SQL Server, puede hacerlo. Deberá importar los CSV con la utilidad de carga de su motor y sustituir las funciones de fecha por sus equivalentes. Tenga presente que MySQL no soporta GROUPING SETS ni la cláusula PIVOT; en ese caso use WITH ROLLUP y expresiones CASE. El diseño del datamart es idéntico.

2.2. Instalación del entorno

Abra una terminal (Símbolo del sistema en Windows, Terminal en macOS o Linux) y ejecute:

```
pip install duckdb

# Verifique la instalacion:
python -c "import duckdb; print(duckdb.__version__)"
```

Debe imprimirse un número de versión 1.0 o superior. Si el comando pip no es reconocido, utilice python -m pip install duckdb.

Cree una carpeta de trabajo llamada lab02 y, dentro de ella, una subcarpeta datos donde copiará los archivos de origen. La estructura debe quedar así:

```
lab02/
|-- datos/
|   |-- ventas_cabecera.csv
|   |-- ventas_detalle.csv
|   |-- productos.csv
|   |-- tiendas.csv
|   |-- clientes.csv
|   |-- promociones.csv
|   |-- inventario_cierre_mensual.csv
+-- lab02_datamart.py      <- el script que usted va a escribir
```

2.3. Descripción de las fuentes

Los archivos entregados simulan el sistema transaccional de MercaAndina, la cadena de supermercados presentada en la sesión de teoría. Corresponden a operaciones registradas entre el 1 de enero de 2025 y el 30 de junio de 2026.

Archivo	Contenido	Filas
ventas_cabecera.csv	Cabecera de cada comprobante: número, tipo, fecha, hora, tienda, documento del cliente y forma de pago.	19 408
ventas_detalle.csv	Línea de detalle de cada comprobante: SKU, cantidad, precio unitario, importe bruto, descuento y promoción aplicada.	66 214
productos.csv	Maestro de productos: SKU, nombre, marca, subcategoría, categoría, línea, unidad, precio de lista y costo unitario.	52
tiendas.csv	Maestro de tiendas: código, nombre, formato, distrito, zona comercial, ciudad, región, fecha de apertura y metros cuadrados.	12
clientes.csv	Maestro de clientes del programa de fidelización: documento, nombre, segmento, tarjeta, distrito y fecha de alta.	600

promociones.csv	Catálogo de campañas comerciales con su vigencia y porcentaje de descuento.	7
inventario_cierre_mensual.csv	Stock de cierre por mes, tienda y producto. Se utiliza únicamente en la actividad propuesta.	9 676

Tabla 1. Archivos de origen entregados con la guía.

Todos los archivos usan coma como separador, punto como separador decimal y codificación UTF-8. Contienen problemas de calidad deliberados: forma parte del ejercicio detectarlos y resolverlos.

3. Paso 1: conexión y carga de las fuentes

Cree el archivo lab02_datamart.py y escriba el siguiente bloque inicial. Le recomendamos ejecutar el script por partes, verificando el resultado de cada paso antes de continuar.

```
import duckdb

# La base se crea como archivo en disco: podrá reabrirla luego sin recargar.
con = duckdb.connect('lab02.duckdb')

# DuckDB lee los CSV directamente. Creamos una vista por cada fuente.
# Importante: forzamos a texto los numeros de documento. Si dejamos que el
# motor los infiera, los tomara como enteros y luego no podremos insertar
# la fila de valor desconocido.
con.execute("""
CREATE OR REPLACE VIEW src_ventas_cab AS
  SELECT * FROM read_csv('datos/ventas_cabecera.csv',
    header=true, types={'nro_documento_cliente': 'VARCHAR'});

CREATE OR REPLACE VIEW src_ventas_det AS
  SELECT * FROM read_csv_auto('datos/ventas_detalle.csv', header=true);

CREATE OR REPLACE VIEW src_productos AS
  SELECT * FROM read_csv_auto('datos/productos.csv', header=true);

CREATE OR REPLACE VIEW src_tiendas AS
  SELECT * FROM read_csv_auto('datos/tiendas.csv', header=true);

CREATE OR REPLACE VIEW src_clientes AS
  SELECT * FROM read_csv('datos/clientes.csv',
    header=true, types={'nro_documento': 'VARCHAR'});

CREATE OR REPLACE VIEW src_promos AS
  SELECT * FROM read_csv_auto('datos/promociones.csv', header=true);

CREATE OR REPLACE VIEW src_inventario AS
  SELECT * FROM read_csv_auto('datos/inventario_cierre_mensual.csv', header=true);
""")

def q(sql):
    """Ejecuta una consulta y muestra el resultado."""
    print(con.execute(sql).df().to_string(index=False))
```

Verifique la carga contando las filas de cada fuente:

```
q("""
SELECT 'ventas_cabecera' AS fuente, COUNT(*) AS filas FROM src_ventas_cab
UNION ALL SELECT 'ventas_detalle', COUNT(*) FROM src_ventas_det
UNION ALL SELECT 'productos', COUNT(*) FROM src_productos
UNION ALL SELECT 'tiendas', COUNT(*) FROM src_tiendas
UNION ALL SELECT 'clientes', COUNT(*) FROM src_clientes
UNION ALL SELECT 'promociones', COUNT(*) FROM src_promos
UNION ALL SELECT 'inventario', COUNT(*) FROM src_inventario
ORDER BY fuente;
""")
```

Los conteos deben coincidir con la columna «Filas» de la Tabla 1. Si alguno difiere, revise la ruta de la carpeta datos antes de continuar.

4. Paso 2: perfilado de las fuentes

Antes de diseñar nada hay que conocer el dato. El perfilado consiste en interrogar a las fuentes para descubrir sus problemas de calidad, porque cada uno de ellos exige una decisión explícita de diseño. Ejecute las cuatro consultas siguientes y anote sus hallazgos: los necesitará en el informe.

4.1. ¿Cómo están escritas las categorías?

```
q("""
SELECT categoria, COUNT(*) AS productos
FROM   src_productos
GROUP BY categoria
ORDER BY categoria;
""")
```

Observará que aparecen doce categorías distintas cuando en realidad son nueve: los valores «Lacteos», «LACTEOS» y « Lacteos» son la misma categoría escrita de tres maneras, y lo mismo ocurre con Abarrotes y Bebidas. Si cargara este maestro tal cual, un informe de venta por categoría mostraría la misma categoría partida en varias filas. Esta es, en pequeño, la razón por la que Inmon exige que un almacén de datos sea integrado.

4.2. ¿Hay claves duplicadas en el maestro de productos?

```
q("""
SELECT sku, COUNT(*) AS veces
FROM   src_productos
GROUP BY sku
HAVING COUNT(*) > 1;
""")
```

Debe encontrar un SKU repetido. Una dimensión no puede tener claves duplicadas: si lo permitiéramos, la unión con la tabla de hechos multiplicaría las filas y los importes quedarían inflados. Lo resolveremos en el paso 4.

4.3. ¿Faltan datos obligatorios?

```
q("""
SELECT sku, nombre_producto, precio_lista, costo_unitario
FROM   src_productos
WHERE  costo_unitario IS NULL;
""")

q("""
SELECT COUNT(*) AS comprobantes_sin_cliente,
       ROUND(100.0 * COUNT(*) /
             (SELECT COUNT(*) FROM src_ventas_cab), 1) AS pct
FROM   src_ventas_cab
WHERE  nro_documento_cliente IS NULL;
""")
```

Dos productos no tienen costo registrado, lo que impedirá calcular su margen. Y cerca del 39 % de los comprobantes no identifica al cliente, algo perfectamente normal en retail: quien paga en efectivo y no presenta su tarjeta no queda identificado. Este segundo hallazgo es el más importante del perfilado, porque determina que necesitemos obligatoriamente una fila de cliente «no determinado» en la dimensión.

¿Por qué no basta con descartar esas ventas? Porque son ventas reales: representan casi cuatro de cada diez comprobantes y una porción equivalente del importe. Excluir las produciría un informe de ventas que no cuadra con la contabilidad. La fila de valor desconocido existe precisamente para conservar el hecho cuando el contexto no se conoce.

4.4. ¿Qué periodo cubren los datos?

```
q("""
SELECT MIN(fecha_emision) AS desde,
       MAX(fecha_emision) AS hasta,
       COUNT(DISTINCT fecha_emision) AS dias_con_venta
FROM   src_ventas_cab;
""")
```

4.5. Registre sus hallazgos

Complete esta tabla en su informe. La última columna es la que se evalúa.

Hallazgo del perfilado	Evidencia (dato observado)	Decisión de diseño que exige
Categorías escritas de forma inconsistente		
SKU duplicado en el maestro de productos		
Productos sin costo unitario		
Comprobantes sin cliente identificado		
Periodo cubierto por los datos		

Tabla 2. Hoja de registro del perfilado (para completar).

5. Paso 3: diseño del esquema estrella

Este paso se resuelve con lápiz y papel, no con SQL. Es el más corto del laboratorio y el que determina todo lo demás.

5.1. Aplique el método de cuatro pasos

Complete la siguiente hoja de diseño antes de escribir una sola línea de código. Debe incluirla en su informe.

Paso	Pregunta que debe responder	Su respuesta
1. Proceso de negocio	¿Qué actividad operativa se quiere medir?	
2. Grano	¿Qué representa exactamente una fila de la tabla de hechos? Redáctelo en una sola frase.	
3. Dimensiones	¿Qué contextos aplican a ese grano? (quién, qué, cuándo, dónde, cómo)	
4. Hechos	¿Qué magnitudes numéricas se miden? Clasifique la aditividad de cada una.	

Tabla 3. Hoja de diseño del método de cuatro pasos (para completar).

Verificación del grano. Antes de continuar, someta su frase a esta prueba: entréguesela a un compañero y pregúntele cuántas filas tendría la tabla de hechos. Si no puede responder con precisión, su grano no está bien declarado. Recuerde que el grano correcto es el más fino que la fuente permita.

5.2. Modelo objetivo

Para que todos trabajemos sobre la misma base en los pasos siguientes, este es el esquema que construiremos. Compárelo con el que usted diseñó y anote en el informe las diferencias

que encuentre y su justificación: no es un error tener un diseño distinto si está bien argumentado.

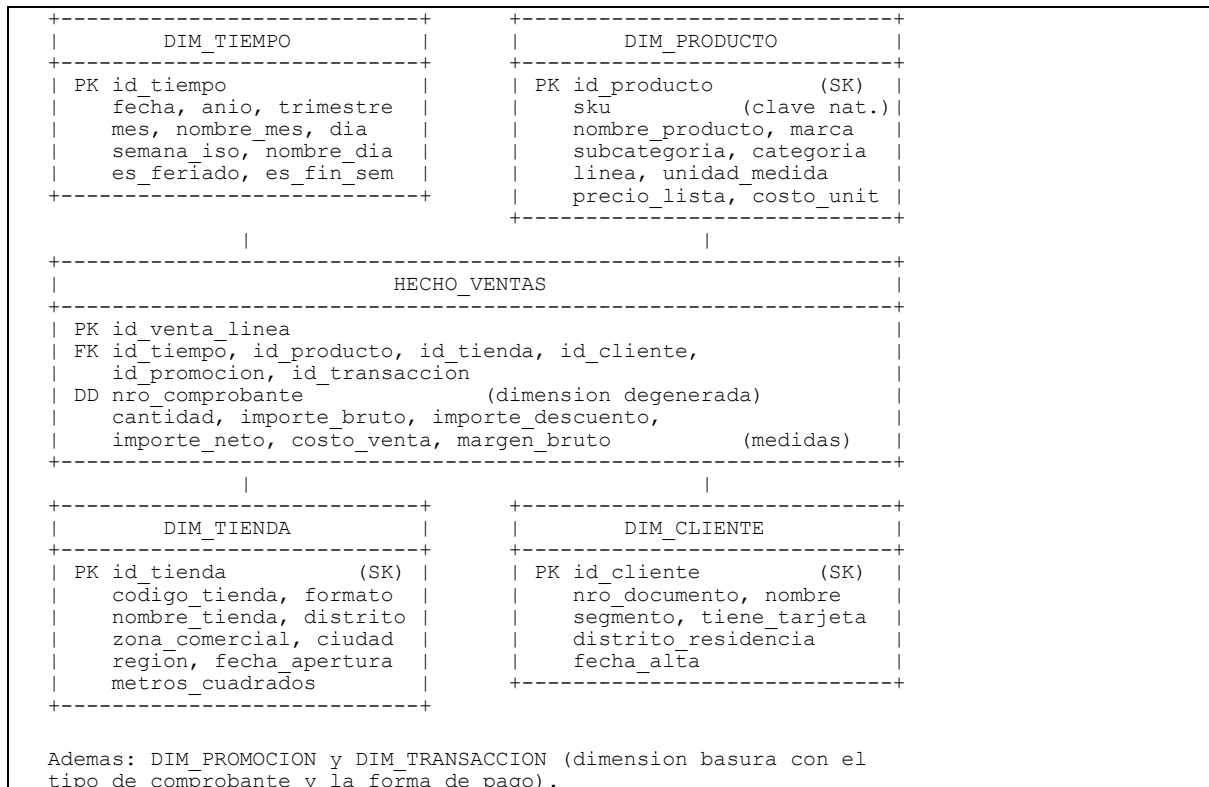


Figura 1. Esquema estrella objetivo del laboratorio.

Note tres decisiones de diseño que conviene comentar:

- **El número de comprobante es una dimensión degenerada.** Vive dentro del hecho y no tiene tabla propia, porque no aporta ningún atributo descriptivo. Sirve para agrupar las líneas de una misma transacción y para calcular el ticket promedio.
- **El tipo de comprobante y la forma de pago van juntos en una dimensión basura.** Son dos indicadores de baja cardinalidad que no justifican dimensiones separadas; combinados producen apenas ocho filas.
- **El porcentaje de margen no aparece entre las medidas.** Es una medida no aditiva: almacenamos margen_bruto e importe_netto y el porcentaje se calcula al consultar, sobre los totales ya agregados.

6. Paso 4: construcción de la dimensión tiempo

La dimensión tiempo se genera por adelantado; nunca se deriva de las fechas presentes en el hecho. La razón es que necesitamos que existan todos los días del calendario, incluso aquellos en que no hubo ninguna venta: sin ellos, un gráfico de serie temporal saltaría los días sin movimiento y los cálculos de promedio diario darían resultados incorrectos.

Generamos el calendario completo de 2025 y 2026, más amplio que el periodo de los datos, para que el datamart admita cargas posteriores sin modificar la dimensión.

```

con.execute("""
CREATE OR REPLACE TABLE dim_tiempo AS
WITH calendario AS (
  SELECT CAST(RANGE AS DATE) AS fecha

```

```
FROM RANGE (DATE '2025-01-01', DATE '2027-01-01', INTERVAL 1 DAY)
)
SELECT CAST(strftime(fecha, '%Y%m%d') AS INTEGER) AS id_tiempo,
       fecha,
       YEAR(fecha) AS anio,
       QUARTER(fecha) AS trimestre,
       'T' || QUARTER(fecha) AS nombre_trimestre,
       MONTH(fecha) AS mes,
       strftime(fecha, '%Y-%m') AS anio_mes,
       CASE MONTH(fecha)
         WHEN 1 THEN 'Enero' WHEN 2 THEN 'Febrero'
         WHEN 3 THEN 'Marzo' WHEN 4 THEN 'Abril'
         WHEN 5 THEN 'Mayo' WHEN 6 THEN 'Junio'
         WHEN 7 THEN 'Julio' WHEN 8 THEN 'Agosto'
         WHEN 9 THEN 'Setiembre' WHEN 10 THEN 'Octubre'
         WHEN 11 THEN 'Noviembre' ELSE 'Diciembre'
       END AS nombre_mes,
       WEEKOFYEAR(fecha) AS semana_iso,
       DAY(fecha) AS dia,
       DAYOFWEEK(fecha) AS dia_semana,
       CASE DAYOFWEEK(fecha)
         WHEN 0 THEN 'Domingo' WHEN 1 THEN 'Lunes'
         WHEN 2 THEN 'Martes' WHEN 3 THEN 'Miercoles'
         WHEN 4 THEN 'Jueves' WHEN 5 THEN 'Viernes'
         ELSE 'Sabado'
       END AS nombre_dia,
       CASE WHEN DAYOFWEEK(fecha) IN (0, 6) THEN 'S' ELSE 'N' END
       AS es_fin_semana,
       -- Feriados nacionales del Peru: atributo del calendario del negocio
       CASE WHEN strftime(fecha, '%m-%d') IN
         ('01-01', '05-01', '06-29', '07-28', '07-29', '08-30',
          '10-08', '11-01', '12-08', '12-25')
         THEN 'S' ELSE 'N' END AS es_feriado
FROM calendario;
""")

q("""SELECT COUNT(*) AS filas, MIN(fecha) AS desde,
      MAX(fecha) AS hasta FROM dim tiempo""")
```

Resultado esperado: 730 filas, del 2025-01-01 al 2026-12-31.

Observe que la clave subrogada de esta dimensión no es un número correlativo, sino la fecha en formato AAAAMMDD. Es la única excepción aceptada a la regla de claves sin significado: facilita el particionamiento físico por fecha y permite depurar las consultas leyendo la clave directamente. También note que `es_feriado` es un atributo del negocio que ningún sistema fuente proporciona: lo aporta el almacén.

7. Paso 5: construcción de las dimensiones restantes

Cada dimensión se construye con tres elementos obligatorios: una clave subrogada generada por el almacén, la clave natural conservada como atributo, y una fila para el valor desconocido con clave -1.

7.1. Dimensión Producto: aquí resolvemos los problemas de calidad

Esta es la dimensión más interesante porque en ella se aplican las tres decisiones que derivaron del perfilado: conformar las categorías, eliminar el duplicado e imputar el costo faltante.

```
con.execute("""
CREATE OR REPLACE TABLE dim_producto AS
WITH limpio AS (
  SELECT sku,
         TRIM(nombre_producto) AS nombre_producto,
         TRIM(marca) AS marca,
         TRIM(subcategoria) AS subcategoria,
         -- CONFORMADO: una sola forma de escribir cada categoria.
         -- Quitamos espacios y normalizamos a formato Titulo.
         UPPER(SUBSTR(TRIM(categoria), 1, 1)) ||
         LOWER(SUBSTR(TRIM(categoria), 2)) AS categoria,
         TRIM(linea) AS linea,
         unidad medida,
```

```
precio_lista,
-- IMPUTACION: si falta el costo, lo estimamos al 72% del precio
-- y lo marcamos, para no ocultar que es un valor calculado.
COALESCE(costo_unitario, ROUND(precio_lista * 0.72, 2))
        AS costo_unitario,
CASE WHEN costo_unitario IS NULL THEN 'S' ELSE 'N' END
        AS costo_imputado,
-- DEDUPLICACION: numeramos las filas de cada SKU y luego
-- nos quedamos solo con la primera.
ROW_NUMBER() OVER (PARTITION BY sku ORDER BY nombre_producto)
        AS rn
FROM src_productos
)
SELECT ROW_NUMBER() OVER (ORDER BY sku) AS id_producto, -- clave subrogada
       sku, nombre_producto, marca, subcategoria, categoria, linea,
       unidad_medida, precio_lista, costo_unitario, costo_imputado
FROM   limpio
WHERE  rn = 1;

-- Fila de valor desconocido: sin ella se pierden hechos en las uniones.
INSERT INTO dim_producto VALUES
(-1, 'N/D', 'No determinado', 'No determinada', 'No determinada',
 'No determinada', 'No determinada', 'N/D', 0, 0, 'N');
""")
```

Verifique que el conformado funcionó:

```
q("""
SELECT categoria, COUNT(*) AS productos
FROM   dim_producto
WHERE  id_producto <> -1
GROUP BY categoria
ORDER BY categoria;
""")
```

Debe obtener nueve categorías, no doce. Compare este resultado con el de la sección 4.1: esa diferencia es, en concreto, lo que significa que un almacén de datos sea «integrado».

7.2. Dimensiones Tienda, Cliente, Promoción y Transacción

Las tres primeras siguen el mismo patrón. La cuarta se construye a partir de las combinaciones que existen en los datos.

```
con.execute("""
CREATE OR REPLACE TABLE dim_tienda AS
SELECT ROW_NUMBER() OVER (ORDER BY codigo_tienda) AS id_tienda,
       codigo_tienda, nombre_tienda, formato, distrito, zona_comercial,
       ciudad, region, fecha_apertura, metros_cuadrados
FROM   src_tiendas;

INSERT INTO dim_tienda VALUES
(-1, 'N/D', 'No determinada', 'No determinado', 'No determinado',
 'No determinada', 'No determinada', 'No determinada', NULL, 0);

CREATE OR REPLACE TABLE dim_cliente AS
SELECT ROW_NUMBER() OVER (ORDER BY nro_documento) AS id_cliente,
       nro_documento, nombre_cliente, segmento, tiene_tarjeta,
       distrito_residencia, fecha_alta
FROM   src_clientes;

-- Esta fila representa la venta sin cliente identificado: casi el 39%.
INSERT INTO dim_cliente VALUES
(-1, 'N/D', 'Cliente no identificado', 'No identificado', 'N',
 'No determinado', NULL);

CREATE OR REPLACE TABLE dim_promocion AS
SELECT ROW_NUMBER() OVER (ORDER BY codigo_promocion) AS id_promocion,
       codigo_promocion, nombre_promocion, tipo_promocion,
       fecha_inicio, fecha_fin, descuento_pct
FROM   src_promos;

INSERT INTO dim_promocion VALUES
(-1, 'N/D', 'No determinada', 'No determinada', NULL, NULL, 0);

-- DIMENSION BASURA: combina indicadores de baja cardinalidad.
CREATE OR REPLACE TABLE dim_transaccion AS
SELECT ROW_NUMBER() OVER (ORDER BY tipo_comprobante, forma_pago)
       AS id_transaccion,
```

```

tipo_comprobante, forma_pago,
CASE WHEN forma_pago LIKE 'Tarjeta%' THEN 'S' ELSE 'N' END
AS es_pago_con_tarjeta
FROM (SELECT DISTINCT tipo_comprobante, forma_pago FROM src_ventas_cab);

INSERT INTO dim_transaccion VALUES (-1, 'N/D', 'No determinada', 'N');
""")

```

Controle el resultado. Las cifras esperadas se muestran en la columna derecha; cada dimensión incluye su fila -1.

```

q("""
SELECT 'dim tiempo' AS dimension, COUNT(*) AS filas FROM dim_tiempo
UNION ALL SELECT 'dim_producto', COUNT(*) FROM dim_producto
UNION ALL SELECT 'dim_tienda', COUNT(*) FROM dim_tienda
UNION ALL SELECT 'dim_cliente', COUNT(*) FROM dim_cliente
UNION ALL SELECT 'dim_promocion', COUNT(*) FROM dim_promocion
UNION ALL SELECT 'dim_transaccion', COUNT(*) FROM dim_transaccion
ORDER BY dimension;
""")

```

Dimensión	Filas esperadas	Composición
dim_tiempo	730	Calendario completo de 2025 y 2026
dim_producto	52	51 productos únicos (52 filas del maestro menos 1 duplicado) + fila N/D
dim_tienda	13	12 tiendas + fila N/D
dim_cliente	601	600 clientes + fila N/D
dim_promocion	8	7 promociones + fila N/D
dim_transaccion	9	2 tipos de comprobante × 4 formas de pago + fila N/D

Tabla 4. Cifras de control de las dimensiones.

8. Paso 6: carga de la tabla de hechos

Con las dimensiones construidas, la carga del hecho consiste en tomar cada línea de detalle del comprobante y reemplazar sus claves naturales por las claves subrogadas correspondientes. Es lo que en la terminología de ETL se denomina búsqueda de claves o key lookup.

Preste atención al tipo de unión empleado en cada caso, porque no es un detalle menor:

- **INNER JOIN con dim_tiempo.** Toda venta tiene fecha y el calendario cubre todo el periodo, así que la unión nunca falla. Si fallara, indicaría un error grave que debe detenernos.
- **LEFT JOIN con las demás dimensiones, combinado con COALESCE(clave, -1).** Si la clave natural no existe o viene vacía, conservamos la fila del hecho y la asociamos a la fila de valor desconocido. Un INNER JOIN aquí eliminaría silenciosamente el 39 % de las ventas.

```

con.execute("""
CREATE OR REPLACE TABLE hecho_ventas AS
SELECT
    ROW_NUMBER() OVER (ORDER BY c.nro_comprobante, d.nro_linea)
    AS id_venta_linea,
    t.id_tiempo,
    COALESCE(p.id_producto, -1) AS id_producto,
    COALESCE(ti.id_tienda, -1) AS id_tienda,
    COALESCE(cl.id_cliente, -1) AS id_cliente,
    COALESCE(pr.id_promocion, -1) AS id_promocion,
    COALESCE(tr.id_transaccion, -1) AS id_transaccion,
    c.nro_comprobante, -- dimension degenerada
    -- Medidas, todas al grano de linea de comprobante:
    d.cantidad,
    d.importe_bruto,
    d.importe_descuento,
    ROUND(d.importe_bruto - d.importe_descuento, 2) AS importe_netto,
    ROUND(d.cantidad * p.costo_unitario, 2) AS costo_venta,
    ROUND(d.importe_bruto - d.importe_descuento
    - d.cantidad * p.costo_unitario, 2) AS margen_bruto

```

```
FROM      src_ventas_det  d
INNER JOIN src_ventas_cab c ON d.nro_comprobante = c.nro_comprobante
INNER JOIN dim_tiempo    t ON t.fecha          = c.fecha_emision
LEFT JOIN dim_producto   p ON p.sku           = d.sku
LEFT JOIN dim_tienda     ti ON ti.codigo_tienda = c.codigo_tienda
LEFT JOIN dim_cliente    cl ON cl.nro_documento = c.nro_documento_cliente
LEFT JOIN dim_promocion  pr ON pr.codigo_promocion = d.codigo_promocion
LEFT JOIN dim_transaccion tr ON tr.tipo_comprobante = c.tipo_comprobante
                        AND tr.forma_pago        = c.forma_pago;
""")
```

9. Paso 7: verificación de la carga

Ningún datamart se entrega sin cuadros de control. Estas tres verificaciones son el mínimo exigible en cualquier proyecto profesional y son las que un auditor le pedirá primero.

9.1. Cuadre de filas y de importes

El número de filas del hecho debe ser exactamente igual al de líneas de detalle del origen, y la suma del importe bruto debe coincidir al céntimo. Si el hecho tiene más filas que el origen, hay una dimensión con claves duplicadas; si tiene menos, se perdieron hechos en una unión.

```
q("""
SELECT COUNT(*) AS filas_hecho,
      (SELECT COUNT(*) FROM src_ventas_det) AS filas_origen,
      ROUND(SUM(importe_bruto), 2) AS bruto_datamart,
      ROUND((SELECT SUM(importe_bruto) FROM src_ventas_det), 2) AS bruto_origen,
      ROUND(SUM(importe_bruto)
        - (SELECT SUM(importe_bruto) FROM src_ventas_det), 2) AS diferencia
FROM    hecho_ventas;
""")
```

Indicador de control	Valor esperado
Filas en hecho_ventas	66 214
Filas en el origen (ventas_detalle)	66 214
Suma del importe bruto	S/ 1 249 553.68
Suma del importe de descuento	S/ 19 870.48
Suma del importe neto	S/ 1 229 683.20
Diferencia contra el origen	0.00

Tabla 5. Cifras de control de la tabla de hechos.

9.2. Búsqueda de hechos huérfanos

Un hecho huérfano es una fila cuya clave foránea no encuentra correspondencia en la dimensión. Después de aplicar COALESCE con -1 no debería existir ninguno. El resultado de las cuatro consultas debe ser cero.

```
q("""
SELECT 'producto' AS dimension, COUNT(*) AS huerfanos
FROM    hecho_ventas h
LEFT JOIN dim_producto p ON h.id_producto = p.id_producto
WHERE   p.id_producto IS NULL
UNION ALL
SELECT 'tienda', COUNT(*) FROM hecho_ventas h
LEFT JOIN dim_tienda t ON h.id_tienda = t.id_tienda
WHERE   t.id_tienda IS NULL
UNION ALL
SELECT 'cliente', COUNT(*) FROM hecho_ventas h
LEFT JOIN dim_cliente c ON h.id_cliente = c.id_cliente
WHERE   c.id_cliente IS NULL
UNION ALL
SELECT 'tiempo', COUNT(*) FROM hecho_ventas h
LEFT JOIN dim_tiempo t ON h.id_tiempo = t.id_tiempo
WHERE   t.id_tiempo IS NULL;
""")
```

9.3. Peso de los valores desconocidos

Conviene medir cuánto del hecho quedó asociado a la fila -1. Es un indicador de calidad que se reporta al negocio, no un error que se oculte.

```
q("""
SELECT COUNT(*) AS lineas_totales,
       SUM(CASE WHEN id_cliente = -1 THEN 1 ELSE 0 END) AS lineas_sin_cliente,
       ROUND(100.0 * SUM(CASE WHEN id_cliente = -1 THEN 1 ELSE 0 END) / COUNT(*), 1) AS pct_sin_cliente
FROM   hecho_ventas;
""")
```

Obtendrá alrededor del 38 % de las líneas sin cliente identificado. Anótelos en su informe: es el dato que permite responder a la gerencia qué porcentaje de la venta puede analizarse por perfil de cliente y cuál no.

10. Paso 8: operaciones OLAP sobre el datamart

El datamart ya está construido. Ahora ejecutaremos, una por una, las operaciones OLAP estudiadas en la teoría. Incluya en su informe una captura del resultado de cada consulta.

10.1. Roll-up: agregación con subtotales

La cláusula ROLLUP genera automáticamente los niveles jerárquicos de agregación. Observe que produce tres tipos de fila: el detalle por región y categoría, el subtotal por región (cuando categoría es NULL) y el total general (cuando ambas son NULL).

```
q("""
SELECT COALESCE(ti.region, 'TOTAL GENERAL') AS region,
       COALESCE(p.categoria, ' Subtotal region') AS categoria,
       ROUND(SUM(h.importe_neto), 2) AS venta_neta,
       ROUND(SUM(h.margen_bruto), 2) AS margen,
       ROUND(100.0 * SUM(h.margen_bruto) / NULLIF(SUM(h.importe_neto), 0), 1) AS pct_margen
FROM   hecho_ventas h
INNER JOIN dim_tienda ti ON h.id_tienda = ti.id_tienda
INNER JOIN dim_producto p ON h.id_producto = p.id_producto
INNER JOIN dim_tiempo t ON h.id_tiempo = t.id_tiempo
WHERE  t.anio = 2026
GROUP BY ROLLUP (ti.region, p.categoria)
ORDER BY region, categoria;
""")
```

Punto clave: el porcentaje de margen se calcula como la división de dos sumas, no como el promedio de los porcentajes de cada fila. Esta es exactamente la regla de las medidas no aditivas vista en teoría. Compruébelo: calcule AVG del porcentaje por producto y compare con el resultado anterior; las cifras no coinciden, y la correcta es la de esta consulta.

10.2. Drill-down: descenso en la jerarquía

Partimos de la venta anual de una categoría y descendemos a mes y subcategoría.

```
q("""
SELECT t.anio_mes, p.subcategoria,
       ROUND(SUM(h.importe_neto), 2) AS venta_neta
FROM   hecho_ventas h
INNER JOIN dim_producto p ON h.id_producto = p.id_producto
INNER JOIN dim_tiempo t ON h.id_tiempo = t.id_tiempo
WHERE  p.categoria = 'Lacteos' AND t.anio = 2026
GROUP BY t.anio_mes, p.subcategoria
ORDER BY t.anio_mes, p.subcategoria;
""")
```

10.3. Slice y dice: acotar el subcubo

El slice fija el valor de una dimensión; el dice restringe rangos en varias a la vez.

```
-- SLICE: fijamos una sola dimension (el año 2026).
q("""
SELECT ti.region, ROUND(SUM(h.importe_net), 2) AS venta_neta
FROM      hecho_ventas h
INNER JOIN dim_tienda ti ON h.id_tienda = ti.id_tienda
INNER JOIN dim_tiempo t ON h.id_tiempo = t.id_tiempo
WHERE     t.año = 2026
GROUP BY ti.region ORDER BY venta_neta DESC;
""")

-- DICE: restringimos zona, categoria y trimestre simultaneamente.
q("""
SELECT ti.zona_comercial, p.categoria, t.nombre_mes,
       ROUND(SUM(h.importe_net), 2) AS venta_neta
FROM      hecho_ventas h
INNER JOIN dim_tienda ti ON h.id_tienda = ti.id_tienda
INNER JOIN dim_producto p ON h.id_producto = p.id_producto
INNER JOIN dim_tiempo t ON h.id_tiempo = t.id_tiempo
WHERE     ti.zona_comercial IN ('Lima Norte', 'Lima Sur')
        AND p.categoria IN ('Lacteos', 'Bebidas')
        AND t.año = 2026 AND t.trimestre = 2
GROUP BY ti.zona_comercial, p.categoria, t.nombre_mes, t.mes
ORDER BY ti.zona_comercial, p.categoria, t.mes;
""")
```

Note el detalle del ORDER BY: incluimos t.mes en el GROUP BY aunque no lo mostramos, porque ordenar por nombre_mes daría un orden alfabético (Abril, Junio, Mayo) en lugar del orden cronológico. Es un error frecuente en los informes.

10.4. Pivot: reorientar los ejes

```
q("""
Pivot (
  SELECT p.categoria, t.nombre_trimestre, h.importe_net
  FROM      hecho_ventas h
  INNER JOIN dim_producto p ON h.id_producto = p.id_producto
  INNER JOIN dim_tiempo t ON h.id_tiempo = t.id_tiempo
  WHERE     t.año = 2026
) ON nombre_trimestre USING ROUND(SUM(importe_net), 2)
GROUP BY categoria
ORDER BY categoria;
""")
```

El resultado presenta las categorías en filas y los trimestres en columnas: es la misma información de una tabla dinámica de Excel, generada en el servidor.

10.5. Comparativo contra el mismo periodo del año anterior

Este es el patrón analítico más solicitado en retail. Requiere una función de ventana.

```
q("""
WITH venta_mensual AS (
  SELECT t.año, t.mes, p.categoria,
         SUM(h.importe_net) AS venta_neta
  FROM      hecho_ventas h
  INNER JOIN dim_tiempo t ON h.id_tiempo = t.id_tiempo
  INNER JOIN dim_producto p ON h.id_producto = p.id_producto
  GROUP BY t.año, t.mes, p.categoria
),
con_anterior AS (
  SELECT año, mes, categoria, venta_neta,
         LAG(venta_neta) OVER (PARTITION BY categoria, mes ORDER BY año)
         AS venta_año_anterior
  FROM venta_mensual
)
SELECT año, mes, categoria,
       ROUND(venta_neta, 2) AS venta_neta,
       ROUND(venta_año_anterior, 2) AS venta_año_anterior,
       ROUND(100.0 * (venta_neta - venta_año_anterior)
             / NULLIF(venta_año_anterior, 0), 1) AS variacion_pct
FROM   con_anterior
WHERE  año = 2026
ORDER BY categoria, mes;
```

""")

Atención a la estructura de esta consulta. El cálculo del LAG se hace en una CTE separada y el filtro por año se aplica después. Si pusiera WHERE anio = 2026 junto con la función de ventana, el filtro se evaluaría antes que la ventana, las filas de 2025 desaparecerían y la columna del año anterior saldría vacía. Es un error muy común y difícil de detectar, porque la consulta se ejecuta sin arrojar ningún mensaje de error.

10.6. Análisis del programa de fidelización

```
q("""
SELECT t.anio_mes,
       ROUND(SUM(h.importe_netto), 2) AS venta_total,
       ROUND(SUM(CASE WHEN c.tiene_tarjeta = 'S'
                      THEN h.importe_netto ELSE 0 END), 2)
       AS venta_fidelizada,
       ROUND(100.0 * SUM(CASE WHEN c.tiene_tarjeta = 'S'
                              THEN h.importe_netto ELSE 0 END)
             / NULLIF(SUM(h.importe_netto), 0), 1) AS pct_fidelizada
FROM   hecho_ventas h
INNER JOIN dim_cliente c ON h.id_cliente = c.id_cliente
INNER JOIN dim_tiempo t ON h.id_tiempo = t.id_tiempo
GROUP BY t.anio_mes
ORDER BY t.anio_mes;
""")
```

10.7. Preguntas de análisis

Responda en su informe, apoyándose en los resultados obtenidos:

1. ¿Qué región concentra la mayor venta neta en 2026 y qué categoría aporta el mayor margen porcentual? ¿Coinciden la región que más vende y la que mejor margen obtiene?
2. En la consulta de fidelización, ¿el porcentaje de venta fidelizada está subestimado o sobrestimado por la existencia de la fila de cliente no identificado? Justifique.
3. ¿Qué ocurriría con los totales de la sección 10.1 si en el paso 6 hubiera usado INNER JOIN en lugar de LEFT JOIN con dim_cliente? Cuantifique el impacto.
4. Los dos productos con costo imputado, ¿afectan de manera significativa el margen reportado por categoría? Diseñe y ejecute una consulta que lo demuestre.

11. Actividad propuesta (40 minutos)

Esta actividad se desarrolla de forma individual al término de la guía dirigida. Su propósito es que usted aplique por su cuenta lo practicado, sobre un segundo proceso de negocio.

11.1. Situación

La gerencia comercial de MercaAndina quiere ir más allá del análisis de ventas. Necesita responder una pregunta que ningún informe actual resuelve: ¿qué categorías rotan bien y cuáles acumulan stock sin venderse, y cómo varía eso entre regiones? Para ello dispone del archivo inventario_cierre_mensual.csv, que registra el stock de cada producto en cada tienda al cierre de cada mes.

Responder esa pregunta exige combinar dos procesos de negocio distintos —la venta y el inventario— en un mismo informe. En la terminología de Kimball, esa operación es un drill-across y sólo es posible si ambos hechos comparten dimensiones conformadas. Usted ya tiene construidas las dimensiones Tiempo, Producto y Tienda: no debe crear ninguna nueva, y ese es precisamente el punto del ejercicio.

11.2. Lo que debe hacer

1. **Declare el grano del nuevo hecho** y determine de qué tipo de tabla de hechos se trata entre los cuatro estudiados en teoría. Justifique su respuesta en dos líneas.
2. **Construya la tabla hecho_inventario** reutilizando exclusivamente las dimensiones ya existentes. Aplique el mismo patrón de búsqueda de claves y de manejo de valores desconocidos que empleó en el paso 6.
3. **Verifique la carga** con los cuadros de control correspondientes: número de filas contra el origen, ausencia de huérfanos y número de cierres mensuales cargados.
4. **Demuestre que el stock es una medida semiaditiva.** Escriba una consulta que sume el stock a lo largo del año y otra que aplique el tratamiento correcto. Explique en su informe por qué la primera cifra no tiene sentido de negocio.
5. **Ejecute el drill-across.** Construya una consulta que muestre, por mes y categoría, las unidades vendidas, el stock de cierre y un índice de rotación definido como unidades vendidas dividido entre stock de cierre. Identifique las tres categorías de menor rotación en el primer semestre de 2026.
6. **Interprete el resultado.** Redacte una recomendación de dos o tres líneas dirigida a la gerencia comercial, fundamentada en las cifras que obtuvo.

11.3. Orientaciones

La estructura del archivo de inventario es la siguiente:

Columna	Descripción
fecha_cierre	Último día del mes al que corresponde el conteo. Une contra dim_tiempo por el campo fecha.
codigo_tienda	Clave natural de la tienda. Une contra dim_tienda.
sku	Clave natural del producto. Une contra dim_producto.
stock_unidades	Unidades en existencia al cierre. Medida semiaditiva.
valorizado_costo	Valor del stock a costo. Medida semiaditiva.

Para el drill-across, la estructura recomendada es construir una CTE por cada hecho, agregando cada una al mismo nivel de detalle (mes y categoría), y unir las después por sus atributos conformados. Nunca una las dos tablas de hechos directamente entre sí: eso produciría un producto cartesiano y cifras infladas.

```
-- Estructura sugerida (complete usted el contenido):  
WITH venta AS (  
    SELECT ... FROM hecho_ventas ... GROUP BY mes, categoria  
),  
stock AS (  
    SELECT ... FROM hecho_inventario ... GROUP BY mes, categoria  
)  
SELECT v.anio_mes, v.categoria, ...  
FROM     venta v  
INNER JOIN stock s ON v.anio_mes = s.anio_mes  
                AND v.categoria = s.categoria  
ORDER BY ...;
```

Cifras de control: el archivo tiene 9 676 filas y contiene 18 cierres mensuales, de enero de 2025 a junio de 2026. Advierta que no todos los productos existen en todas las tiendas, lo que es normal: la dispersión del cubo, discutida en teoría, aquí se hace visible.

11.4. Criterios de evaluación de la actividad

Criterio	Puntos
Grano declarado con precisión y tipo de tabla de hechos correctamente identificado y justificado	4
Tabla hecho_inventario construida reutilizando las dimensiones conformadas, sin crear dimensiones nuevas	4
Cuadros de control ejecutados y documentados	3
Demostración de la semiaditividad del stock, con explicación de negocio	4
Consulta de drill-across correcta y sin duplicación de filas	3
Interpretación y recomendación fundamentada en las cifras	2
Total	20

Tabla 6. Rúbrica de la actividad propuesta.

12. Evaluación del laboratorio

Componente	Criterio de logro	Peso
Perfilado (sección 4)	Identifica los cinco hallazgos y asocia a cada uno una decisión de diseño pertinente.	15 %
Diseño dimensional (sección 5)	Declara el grano en una frase precisa y al nivel atómico; identifica dimensiones y clasifica la aditividad de las medidas.	20 %
Construcción (secciones 6 a 8)	Dimensiones con clave subrogada y fila de valor desconocido; hecho cargado sin pérdida de filas.	20 %
Verificación (sección 9)	Ejecuta y documenta los tres cuadros de control con los valores esperados.	15 %
Consultas OLAP (sección 10)	Ejecuta las seis operaciones y responde correctamente las preguntas de análisis.	15 %
Actividad propuesta (sección 11)	Según la rúbrica de la Tabla 6.	15 %

Tabla 7. Distribución de la calificación del laboratorio.

12.1. Advertencia sobre la originalidad

El laboratorio es individual. Se admite la consulta entre compañeros para resolver dudas conceptuales, pero el script, el informe y las respuestas de análisis deben ser de elaboración propia.

12.2. Errores frecuentes que conviene evitar

Error	Cómo se detecta	Cómo se corrige
Usar INNER JOIN con dim_cliente en la carga del hecho	El hecho tiene menos filas que el origen y el importe no cuadra	Usar LEFT JOIN con COALESCE(clave, -1)
No eliminar el SKU duplicado del maestro	El hecho tiene más filas que el origen	Deduplicar con ROW_NUMBER() antes de crear la dimensión
No conformar las categorías	La misma categoría aparece varias veces en los informes	Normalizar con TRIM y ajuste de mayúsculas al construir la dimensión
Calcular el margen porcentual como promedio de porcentajes	El total no coincide con margen dividido entre venta	Dividir la suma del margen entre la suma del importe neto
Filtrar por año en la misma consulta donde se calcula el LAG	La columna del año anterior sale vacía	Calcular la ventana en una CTE y filtrar después
Ordenar los meses por su nombre	Aparecen en orden alfabético en lugar de cronológico	Incluir el número de mes en el GROUP BY y ordenar por él

Tabla 8. Errores frecuentes en este laboratorio.

13. Referencias

- Kimball, R. y Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3.^a ed. Wiley. Capítulos 1 a 3.
- Joyanes, L. (2019). Inteligencia de negocios y analítica de datos: Una visión global de Business Intelligence & Analytics. Alfaomega Grupo Editor.
- Pérez, M. (2015). Business Intelligence: Técnicas, herramientas y aplicaciones. Alfaomega.
- Kumar, A. (2022). Business Intelligence Demystified. BPB Publications.
- Documentación oficial de DuckDB: <https://duckdb.org/docs> — secciones SQL Introduction, Aggregates y Window Functions.